SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Operating Systems COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Dr VIMAL V R

QUESTIONS: 5 **Total Marks:25**

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I **SIVA SUBRAMAMIYAM B/192572089** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare **Contiguous and Linked file allocation methods** in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between **Linked and Indexed file allocation** with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5
3	Compare **FCFS and SSTF disk scheduling algorithms** based on seek time, fairness, andthepossibilityofstarvation.Discusswhichalgorithmperformsbetterunder heavy disk load.	5
4	Analyzethedifferencesbetween**SCANandC-SCANdiskschedulingalgorithms**in terms of head movement, response time, and uniformity of service. Which algorithm Is more suitable for real-time systems and why?	5
5	Compare **LOOK and C-LOOK disk scheduling algorithms** with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5
6	Compare Contiguous, Linked, and Indexed file allocation methods in terms of file growth, block allocation overhead, direct access, and fragmentation. A multimedia server stores files whose sizes frequently increase after creation. Evaluate which allocation method would be most suitable and justify your choice based on performance and scalability.	5
7	Compare Indexed and Contiguous allocation for a database system that performs frequent random reads and writes on large files. Analyze their impact on access time, metadata overhead, disk-space utilization, and file expansion. Determine which method would provide better overall performance and justify your answer.	5
8	A disk-intensive server receives hundreds of I/O requests simultaneously, with requests distributed across the entire disk. **Compare SSTF, SCAN, and LOOK** in terms of average seek time, starvation, fairness, and throughput. **Evaluate which algorithm would provide the best balance between performance and fairness under sustained heavy workload.**	5
9	A real-time monitoring system generates disk requests continuously from multiple processes. **Compare FCFS, SCAN, C-SCAN, and SSTF** based on response-time predictability, starvation risk, fairness, and head movement. **Identify the most appropriate algorithm for the system and justify why predictable response time is more important than minimum average seek time.**	5

10	A large scientific-data server performs both sequential and random access to files stored on an HDD. **Compare Linked allocation and Indexed allocation together with SSTF and LOOK disk scheduling** as a combined file-management strategy. Analyze how the choice of file allocation affects disk access patterns and how the scheduling algorithm influences overall performance. **Recommend the best combination for minimizing access delay while maintaining reliable file access, with suitable justification.**	**5**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Contiguous vs Linked File Allocation

The question asks us to compare the methods based on access speed, memory utilization, fragmentation, and suitability for sequential access.

Feature	**Contiguous Allocation**	*Linked Allocation*
Access speed	Very fast because blocks are adjacent and direct access is possible.	*Slower because blocks may be scattered and pointers must be followed.*
Memory utilization	Requires consecutive free blocks, so utilization can be affected by external fragmentation.	*Uses scattered free blocks efficiently and avoids external fragmentation.*
Fragmentation	Suffers from external fragmentation.	*No external fragmentation, but pointer space creates overhead.*
File growth	Difficult if adjacent space is unavailable.	*Easy because new blocks can be linked anywhere on the disk.*
Sequential access	**Excellent because consecutive blocks can be accessed continuously.**	***Also suitable, but pointer traversal introduces some overhead.***

Conclusion:

For sequential file access, Contiguous Allocation is more suitable because its blocks are stored consecutively, resulting in lower seek time and faster sequential reading. However, Linked Allocation is better when files frequently grow.

2. Linked vs Indexed File Allocation

The question focuses on random access, storage overhead, reliability, and large files.

Feature	Linked Allocation	Indexed Allocation
Random access	Poor because pointers must be followed from the beginning.	**Excellent because the index block contains addresses of file blocks.**
Storage overhead	Each data block requires a pointer.	**Requires additional index block(s).**
Reliability	A damaged pointer can make the remaining part of the file inaccessible.	**More reliable for access because block addresses are maintained in the index.**
Large files	Can support large files, but traversal becomes slow.	**Efficient for large files, especially with multi-level indexing.**
Access speed	**Better for sequential access.**	**Better for direct/random access.**

Conclusion:

Indexed Allocation is more efficient for large files, particularly when random access is required. Its index structure allows the operating system to locate file blocks directly without traversing a chain of pointers.

3. FCFS vs SSTF Disk Scheduling

The question asks for comparison based on seek time, fairness, starvation, and heavy disk load.

Feature	FCFS	SSTF
Working principle	Services requests in their arrival order.	**Services the request closest to the current head position.**
Seek time	Can be high because requests may be far apart.	**Usually lower because nearby requests are selected.**
Fairness	Very fair because requests are handled in order.	**Less fair because distant requests may wait.**
Starvation	No starvation.	**Starvation is possible.**
Heavy load	**Performance may decrease due to unnecessary head movement.**	**Usually provides better average seek time under heavy load.**

Conclusion:

SSTF generally performs better under heavy disk load because it minimizes immediate head movement and reduces average seek time. However, FCFS is fairer and guarantees that requests are processed in arrival order. SSTF should therefore be used when performance is more important than strict fairness.

4. SCAN vs C-SCAN Disk Scheduling

The question compares them using head movement, response time, and uniformity of service, and asks which is suitable for real-time systems.

Feature	SCAN	C-SCAN
Movement	Head moves in both directions like an elevator.	**Head services requests in one direction only.**
Response time	Good, but response time can vary depending on direction.	**More uniform response time.**
Service uniformity	Less uniform because requests in the current direction get serviced first.	**More uniform because all requests receive service during a single sweep.**
Head movement	Efficient because requests are served in both directions.	**May involve extra movement when returning to the beginning.**
Suitable use	**General-purpose systems with heavy loads.**	**Systems requiring more predictable service.**

Conclusion:

C-SCAN is more suitable when predictable response time and uniform service are important, such as real-time or time-sensitive systems. It treats requests more uniformly by servicing them in one direction.

5. LOOK vs C-LOOK Disk Scheduling

The question asks for efficiency, total head movement, practical implementation, and performance conditions.

Feature	LOOK	C-LOOK
Working	Moves in one direction, reverses after reaching the last pending request.	**Moves in one direction and jumps back to the first pending request.**
Efficiency	Efficient because it does not travel to the physical disk end unnecessarily.	**Efficient and provides more uniform service.**
Head movement	Usually lower than SCAN.	**Can provide predictable movement but includes a return jump.**
Response time	Can vary based on request location.	**More uniform because requests are handled in one direction.**
Practical implementation	**Relatively simple.**	**Slightly more complex due to circular movement.**

Conclusion:

LOOK provides better performance when minimizing total head movement is the main objective, especially when requests are concentrated in a limited region. C-LOOK is preferable when uniform response time is more important, particularly under heavy workloads.

6. Contiguous vs Linked vs Indexed Allocation

Feature	Contiguous	Linked	Indexed
File growth	Difficult if adjacent blocks are unavailable.	Easy; blocks can be added anywhere.	**Easy; new blocks can be added and recorded in the index.**
Allocation overhead	Very low.	Pointer required in each block.	**Index block requires extra storage.**
Direct access	Excellent.	Poor.	**Excellent.**
Fragmentation	External fragmentation possible.	No external fragmentation.	**No external fragmentation.**
Performance	**Very high for sequential and direct access.**	**Good for sequential access.**	**Good for direct/random access.**

Evaluation:

A multimedia server frequently receives files that increase in size after creation. Contiguous allocation may require relocation if there is no free space immediately after the file. Linked allocation allows easy expansion but has slower access because of pointer traversal.

Conclusion:

Indexed Allocation is the most suitable overall choice for a multimedia server with frequently growing files when both scalability and direct access are important. It allows additional blocks to be allocated without requiring the entire file to be contiguous.

7. Indexed vs Contiguous Allocation for a Database

The question considers a database with frequent random reads/writes on large files and asks about access time, metadata, disk utilization, and expansion.

Feature	**Contiguous Allocation**	***Indexed Allocation***
Random access	Very fast because block locations can be calculated.	*Very fast because the index contains block addresses.*
Metadata overhead	Very low.	*Higher because index blocks are required.*
Disk utilization	Can suffer from external fragmentation.	*Better flexibility in using available blocks.*
File expansion	Difficult when adjacent space is unavailable.	*Easier because additional blocks can be allocated separately.*
Large files	**Excellent access performance but difficult to expand.**	***Good performance with better scalability.***

Analysis:

For a database, random access is important. Both methods provide fast access, but contiguous allocation requires consecutive disk space. For large files that frequently grow, this can become a major limitation.

Conclusion:

Indexed Allocation provides better overall performance for a large, frequently updated database because it supports direct/random access while allowing files to expand without requiring contiguous free space. The additional index storage is the main disadvantage.

8. SSTF vs SCAN vs LOOK Under Heavy Workload

The question describes a disk-intensive server receiving hundreds of requests simultaneously across the entire disk.

Feature	SSTF	SCAN	LOOK
Average seek time	Usually very low.	Moderate and controlled.	**Usually low.**
Starvation	Possible for distant requests.	Very low risk.	**Very low risk.**
Fairness	Lower.	Good.	**Good.**
Throughput	High in many cases.	Good and stable.	**High with less unnecessary movement.**
Heavy workload	**Efficient but can favor nearby requests.**	**Handles heavy workloads fairly.**	**Provides a good balance between efficiency and fairness.**

Evaluation:

SSTF can reduce average seek time, but under sustained heavy workload, requests located far from the current head position may repeatedly be postponed. SCAN provides better fairness by sweeping across the disk.

LOOK improves upon SCAN by reversing at the last pending request instead of travelling all the way to the disk boundary.

Conclusion:

LOOK provides the best balance between performance and fairness for this workload. It reduces unnecessary head movement compared with SCAN while avoiding much of the starvation risk associated with SSTF.

9. FCFS vs SCAN vs C-SCAN vs SSTF for Real-Time Monitoring

The question focuses on response-time predictability, starvation, fairness, and head movement.

Feature	FCFS	SCAN	C-SCAN	SSTF
Predictability	High	Good	Very good	**Low**
Starvation	No	Very low	Very low	**Possible**
Fairness	Excellent	Good	Very good	**Lower**
Head movement	Can be high	Controlled	Controlled	**Usually low**
Real-time suitability	**Moderate**	**Good**	**Excellent**	**Poorer**

For a real-time monitoring system, predictability is more important than minimum average seek time. A scheduling algorithm that occasionally gives extremely long waiting times can cause monitoring information to become outdated.

Conclusion:

C-SCAN is the most appropriate algorithm because it provides relatively uniform waiting time and fair service to requests across the disk. Although SSTF may achieve a lower average seek time, it can starve requests that are far from the current head position. Therefore, C-SCAN is preferable when predictable response is critical.

The question asks for a combined comparison of Linked and Indexed allocation with SSTF and LOOK, considering both sequential and random access.

File Allocation

Linked Allocation:

Suitable mainly for sequential access.

File blocks can be located anywhere on the disk.

Easy file expansion.

Random access is slower because pointers must be followed.

Indexed Allocation:

Provides direct/random access through an index block.

Suitable for large scientific datasets requiring random reads.

File expansion is easier than contiguous allocation.

Has additional index-block overhead.

Disk Scheduling

SSTF:

Selects the request closest to the current head.

Reduces average seek time.

Can cause starvation of distant requests.

LOOK:

Services requests in one direction until the last pending request.

Then reverses direction.

Reduces unnecessary head movement.

Provides better fairness than SSTF.

Recommended Combination

For a scientific-data server performing both sequential and random access, Indexed Allocation + LOOK is the better overall combination.

Indexed allocation provides efficient random access and allows large files to expand. LOOK reduces unnecessary disk-head movement while providing more predictable and fair service than SSTF.

Conclusion:

Indexed Allocation with LOOK scheduling provides a good balance between access speed, scalability, fairness, and reliable file access. It is especially appropriate when large scientific files are accessed using both sequential and random operations.